Guilherme Henrique Bernardi

Gustavo Franco Pereira

Victor Hugo Camargo

Manutenção Predial

Limeira – SP

2026

SUMÁRIO

INTRODUÇÃO 3
OBJETIVO 3
Objetivos Específicos 3
RESULTADOS ESPERADOS 4
REQUISITOS 4
Requisitos Funcionais 4
Requisitos Não Funcionais 5
Priorização 6
METODOLOGIA (SCRUM) 7
Por que utilizar o Scrum no projeto "PredialFix"? 7

INTRODUÇÃO

A manutenção predial eficiente é um dos pilares para o bom funcionamento de instituições educacionais de grande porte. No cenário atual do SENAI, a alta demanda por reparos, que variam de trocas simples de lâmpadas a manutenções hidráulicas complexas, encontra um gargalo na falta de rastreabilidade e na comunicação lenta. O projeto PredialFix surge como a solução tecnológica para este desafio. Focado no desenvolvimento de uma infraestrutura de Back-End, o sistema visa transformar o caos das solicitações em um fluxo de trabalho organizado, oferecendo transparência total aos usuários e uma API segura capaz de priorizar o que é realmente urgente.

OBJETIVO

Desenvolver o Back-End da plataforma PredialFix, focado em uma infraestrutura de servidor robusta e em uma API segura que centralize a gestão de manutenção predial do SENAI, garantindo transparência total desde a abertura do chamado até a sua resolução final.

Objetivos Específicos

- **Gestão de Acesso e Segurança:** Sistema de autenticação seguro que permite a identificação de funcionários e alunos, garantindo a integridade dos dados e o registro de quem abriu cada solicitação.
- **Abertura e Categorização de Chamados:** Interface para descrever o problema (hidráulico, elétrico, estrutural), anexar evidências e localizar o incidente (bloco/sala).
- **Fluxo de Trabalho Transparente (Workflow):** Lógica de estados do chamado (Aberto, Em Análise, Em Execução, Concluído) que permite ao usuário acompanhar cada etapa em tempo real, eliminando a falta de feedback.
- **Algoritmo de Priorização de Urgências:** Classificação automática da criticidade dos pedidos — vazamentos graves têm prioridade sobre lâmpadas queimadas — organizando a fila de trabalho da equipe de manutenção de forma eficiente.
- **Histórico e Relatórios de Performance:** Armazenamento completo de dados para que a gestão visualize o tempo médio de atendimento e os problemas mais frequentes, facilitando decisões estratégicas de infraestrutura.

RESULTADOS ESPERADOS

A implementação do PredialFix deverá transformar a gestão de manutenção predial do SENAI, substituindo o atual modelo por um sistema preditivo e de alta visibilidade. Espera-se que o projeto proporcione uma redução drástica no tempo de resposta aos incidentes, uma vez que a infraestrutura de Back-End permitirá a triagem e priorização automatizada das demandas. A transparência será o resultado mais perceptível para a comunidade escolar; ao permitir que alunos e funcionários acompanhem em tempo real cada etapa do processo, da abertura à resolução, a plataforma eliminará a incerteza e as reclamações por falta de feedback. Por fim, a segurança garantida pela API autenticada assegurará a integridade dos dados, gerando um histórico confiável que servirá de base para auditorias e decisões estratégicas da diretoria.

REQUISITOS

Requisitos Funcionais

1. **RF01 - Autenticação e Gestão de Perfis:** O sistema deve possuir um módulo de autenticação seguro que diferencie, no mínimo, dois níveis de acesso: **Usuário Comum** (funcionários/alunos que apenas abrem e acompanham seus chamados) e **Responsável/Técnico** (equipe de manutenção/chefes que atualizam e gerenciam os chamados).

2. **RF02 - Abertura de Chamados e Priorização:** O sistema deve disponibilizar um endpoint para registro de novos chamados, sendo obrigatório o envio das seguintes informações: Tipo de problema (Elétrica, Hidráulica, Outros), Descrição detalhada da ocorrência, Localização exata e Nível de Urgência.

3. **RF03 - Máquina de Estados (Workflow):** O sistema deve gerenciar o ciclo de vida do chamado, permitindo a transição estrita entre os seguintes status: *Aberto* ➔ *Em Análise* ➔ *Em Execução* ➔ *Concluído*.

4. **RF04 - Filtragem e Listagem:** O sistema deve permitir que os usuários filtrem e visualizem a lista de chamados com base em seus status atuais, localização e nível de urgência.

5. **RF05 - Regra de Negócio de Atualização:** O sistema deve bloquear a alteração de status de um chamado (ex: mover para "Em Execução") caso o usuário logado não tenha o perfil de "Responsável" ou não seja o chefe do setor correspondente.

6. **RF06 - Histórico de Manutenção por Unidade:** O sistema deve armazenar e permitir a consulta de todo o histórico de serviços já realizados, filtráveis por um espaço específico ou área comum (ex: "Laboratório de Informática 3"), para facilitar a análise de reincidências.

7. **RF07 - Transparência e Notificações (Feedback):** O sistema deve disparar atualizações de progresso para o criador do chamado a cada mudança de status (ex: "Em Análise", "Técnico a caminho", "Serviço Concluído"), garantindo a total transparência exigida pelo Senai.

Requisitos Não Funcionais

1. **RNF01 - (Segurança e Autenticação):** A comunicação com a API deve ser criptografada via HTTPS. A autenticação das requisições deve ser feita utilizando tokens seguros (como o padrão JWT - JSON Web Tokens), garantindo que apenas usuários logados acessem os dados.

2. **RNF02 - (Desempenho):** O servidor back-end deve ser capaz de processar as requisições de consulta e abertura de chamados com um tempo de resposta médio inferior a 2 segundos, mesmo lidando com as centenas de solicitações mensais da unidade.

3. **RNF03 - (Disponibilidade):** O sistema deve operar com alta disponibilidade (meta de *uptime* de 99%), garantindo que problemas estruturais possam ser relatados por alunos e funcionários a qualquer momento de funcionamento do Senai.

4. **RNF04 - (Arquitetura e Integração):** O back-end deve ser construído seguindo os padrões de uma API RESTful (ou arquitetura semelhante), retornando dados estruturados em formato JSON, para permitir a futura e fácil integração com o aplicativo mobile do PredialFix.

5. **RNF05 - (Confiabilidade / Integridade de Dados):** O sistema deve utilizar um banco de dados relacional (ex: PostgreSQL, MySQL) robusto, garantindo a integridade referencial entre os chamados, os usuários e os locais do prédio, sem perda de dados em caso de falhas nas transações.

6. **RNF06 - (Documentação):** A API desenvolvida deve ser documentada de forma clara (utilizando ferramentas como *Swagger/OpenAPI*), descrevendo rotas, parâmetros e respostas esperadas, facilitando o trabalho da equipe que for desenvolver a interface (Front-End/App).

Priorização

- MUST HAVE (DEVE TER)
- SHOULD HAVE (DEVERIA TER)
- COULD HAVE (PODERIA TER)
- WON´T HAVE (NÃO TERÁ)

REQUISITO	PRIORIZAÇÃO	EXPLICAÇÃO
RF01, RF02, RF03	**Must have**	Essenciais para a funcionalidade básica do sistema: autenticação, abertura de chamados e gerenciamento do fluxo de trabalho.
RNF01, RNF05	**Must have**	Críticos para segurança dos dados e integridade das informações no sistema.
RF05, RF07	**Must have**	Garantem regras de negócio essenciais e transparência no acompanhamento dos chamados.
RF04, RF06	**Should have**	Importantes para facilitar a gestão e consulta de chamados, permitindo análise de reincidências.
RNF02, RNF03	**Should have**	Desempenho e disponibilidade são importantes para a experiência do usuário e confiabilidade do sistema.
RNF04, RNF06	**Could have**	Facilitam integração futura e manutenção, mas não são bloqueadores para o funcionamento inicial.

METODOLOGIA (SCRUM)

O Scrum é um framework (ou estrutura de trabalho) ágil focado na gestão e no desenvolvimento de projetos complexos. Em vez de tentar construir todo o sistema de uma só vez e entregá-lo apenas no final de meses de trabalho, o Scrum divide o projeto em ciclos curtos e contínuos chamados **Sprints** (que geralmente duram de 1 a 4 semanas).

Ao final de cada Sprint, a equipe entrega uma parte do produto que já funciona e agrega valor. Ele é baseado em três pilares fundamentais: **Transparência**, **Inspeção** e **Adaptação**, e organiza a equipe através de papéis bem definidos (Product Owner, Scrum Master e Desenvolvedores) e cerimônias rápidas (Planejamento, Reunião Diária, Revisão e Retrospectiva).

Por que utilizar o Scrum no projeto "PredialFix"?

O cenário do SENAI exige uma solução rápida, robusta e que resolva o problema da "falta de transparência e demora no atendimento". O Scrum é a escolha ideal para esse desafio pelos seguintes motivos:

- **Entregas Incrementais e Rápidas (Sprints):** Sua equipe não precisa entregar a API inteira de uma vez. Vocês podem usar as Sprints para entregar as *Funcionalidades Essenciais* em partes. Por exemplo: a Sprint 1 pode focar apenas na "Gestão de Usuários" (Autenticação no Laravel). A Sprint 2 foca na "Abertura de Chamados" e no banco de dados MySQL. Isso permite que o SENAI comece a testar o núcleo do sistema muito mais rápido.
- **Priorização Inteligente (Product Backlog):** O desafio fala em "priorizar urgências". No Scrum, os requisitos (as 5 funcionalidades essenciais) formam o *Product Backlog*, uma lista ordenada. O que é mais crítico para o SENAI resolver hoje? Provavelmente o registro do problema e o Workflow. O Scrum garante que a equipe foque primeiro no que traz mais impacto para os funcionários e alunos.
- **Transparência e Alinhamento Constante:** O problema central do SENAI é a falta de transparência nos chamados. Ironicamente, o Scrum resolve a falta de transparência no *desenvolvimento*. Com as reuniões diárias (Daily Scrum), a equipe de Back-End saberá exatamente quem está cuidando da validação de dados (Request Validation), quem está criando as rotas RESTful e quem está travado em algum problema com o Eloquent ORM.

- **Adaptação às Regras de Negócio:** Modelar os status dos chamados (Aberto -> Em Análise -> Em Execução -> Concluído) e a lógica de quem pode atualizar o quê (multi-nível) pode gerar dúvidas durante o desenvolvimento. O Scrum permite que, ao final de cada Sprint (na Revisão), a equipe valide com os interessados se a API está se comportando como esperado, permitindo ajustes rápidos antes de avançar para as "Notificações de Progresso" ou "Histórico".
- **Foco na Qualidade Técnica:** Como há requisitos rigorosos (validação, respostas em JSON, documentação clara), as Sprints permitem que a equipe estabeleça uma "Definição de Pronto" (Definition of Done). Uma funcionalidade só avança se a rota estiver criada, testada, validada e com o código devidamente subido no GitHub.

Ao usar o Scrum, o desenvolvimento do PredialFix deixa de ser uma caixa preta demorada e passa a ser um processo previsível, focado em resolver a dor real do SENAI etapa por etapa.